

Aula 5: Comandos Repetitivos

Luís Felipe

UFF

08 de Setembro de 2025

Comandos condicionais e de repetição

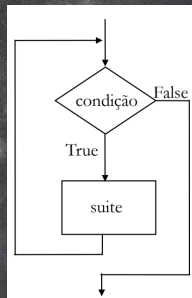
- Até aqui, vimos como escrever programas que executam comandos de forma **sequencial** e tomam decisões com base em **condições**.
- Agora, veremos como **repetir blocos de comandos** várias vezes, quando necessário, para alcançar o resultado desejado.
- **Exemplo:** Programa que imprime todos os números de 1 até 100.

```
print(1)
print(2)
# repete 97 vezes comandos tais como acima
print(100)
```

Estrutura de Repetição Indefinida - While

- A estrutura de repetição **while** é utilizada quando se deseja executar um **bloco** (também chamado de **suíte**) enquanto uma condição for **verdadeira**.

```
1 while <condição>:  
2     <suíte>
```



Estrutura de Repetição Indefinida - While

Exemplo:

```
1 indice = 1                # inicializa variável
2 while indice <= 10:        # testa condição
3     print(indice, end=" ")  # escreve o valor seguido de espaço
4     indice = indice + 1     # incrementa a variável
5 print()                   # pula linha no final
```

Saída esperada:

```
1 1 2 3 4 5 6 7 8 9 10
```

Exemplo com entrada arbitrária:

```
1 a = int(input())
2 indice = 1
3 while indice <= a:
4     print(indice, end = " ")
5     indice += 1          # Outra forma de incrementar a variável
6 print()
```

Obs.: `variavel = variavel + i` é o mesmo que `variavel += i`

Cálculo fatorial

- **Fatorial** de um inteiro n não negativo contabiliza o **número de ordenações** que n elementos possuem. Esse valor é $n!$
 - ▶ $3! = 6$.
 - ▶ Considerando 3 elementos, a, b, c , temos as seguintes ordenações $\{abc, acb, bac, bca, cab, cba\}$.
- $n! = 1.2.3. \dots .(n-2).(n-1).n$
- $1! = 1$

Exercício: Receba um inteiro de entrada e calcule seu fatorial.

```
1 num = int(input("Digite um valor inteiro e positivo: "))
2 i = 1
3 fat = 1
4 while i <= num:
5     fat = fat*i
6     i += 1
7 print("O fatorial de ", num, "=", fat)
```

Obs.: `variavel = variavel*i` é o mesmo que `variavel *= i`

Comportamento do while

1. O que acontece se a condição for falsa na primeira vez?

```
1 while a != a:  
2     a = a + 1
```

Resposta: Ele nunca entra na repetição (no laço).

2. O que acontece se a condição for sempre verdadeira?

```
1 while a == a:  
2     a = a + 1
```

Resposta: Ele entra na repetição e nunca sai (laço infinito).

Cálculo do mdc

- Dado dois inteiros positivos a e b , o **máximo divisor comum** de a e b , denotado por $\text{mdc}(a, b)$, é o maior número inteiro positivo que divide simultaneamente a e b .

► $\text{mdc}(240, 36) = 12$.

$$240 = 36 \times 6 + 24$$

$$36 = 24 \times 1 + 12$$

$$24 = 12 \times 2 + 0$$

```
1 a = int(input())
2 b = int(input())
3 a1 = a
4 b1 = b
5
6 while a%b != 0:
7     c = a%b
8     a = b
9     b = c
10 print("mdc(", a1, b1, ") é: ", b)
```

Listas em Python

- Uma **lista** em Python é uma estrutura que armazena vários dados, que podem ser de um mesmo tipo ou não.
- O acesso a um dado específico da lista é feito por meio da sua posição (**índice**).
- Uma lista é criada com a construção:
[dado1, dado2, ..., dadon].
- Os índices começam em 0, pois essa convenção é herdada de linguagens como C, onde o índice representa a **distância** (ou deslocamento) do primeiro elemento da lista.

Exemplo:

```
1 >>> a = [1, "ola", 2]
2 >>> type(a)
3 <class 'list'>
4 >>> a[0]
5 1
6 >>> a[1]
7 'ola'
8 >>> a[2]
9 2
```

```
1 >>> a = [1, "ola", 2]
2 >>> type(a[0])
3 <class 'int'>
4 >>> type(a[1])
5 <class 'str'>
```

Operações básicas com listas

Seja uma lista com nome `lista`.

- `append(valor)`: adiciona `valor` ao final da lista.
- `len(lista)`: retorna o número de elementos da `lista`.
- `valor in lista`: retorna `True` se `valor` estiver na `lista`, caso contrário, `False`.
- `remove(valor)`: remove a primeira ocorrência de `valor` na lista.
- `pop()`: remove o último elemento da lista.
`pop(indice)`: remove o elemento da posição `indice`.

Exemplo:

```
1 lista = [10, 20, 30]
2 lista.append(40)      # [10, 20, 30, 40]
3 print(len(lista))     # 4
4 print(20 in lista)    # True
5 lista.remove(10)      # [20, 30, 40]
6 x = lista.pop()       # x = 40, lista = [20, 30]
```

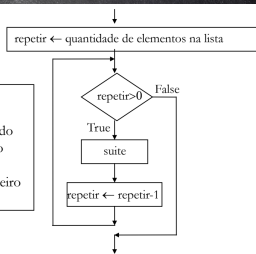
Obs.: Veremos melhor o conceito de listas nas próximas aulas.

Estrutura de Repetição Definida - For

- A estrutura de repetição **for** é utilizada quando se deseja executar um mesmo comando (suite) um número específico de vezes, enumerado em uma lista de valores.

```
1 for <var> in <lista de valores>:  
2     <comando>
```

A cada ciclo da repetição, <var> assume o valor referenciado pela posição do elemento na lista. Isto é, o valor referenciado progressivamente do primeiro até o último elemento.



Estrutura de Repetição Definida (for)

Exemplo:

```
1 lista = [3, 4, 5, 6, 7, 8, 9]
2 for item in lista:           # item percorre a lista iterativamente
3     print(item, end=" ")     # imprime cada item seguido de um espaço
4 print()                     # pula linha ao final
```

Saída esperada:

```
1 3 4 5 6 7 8 9
```

A função range() em Python

A função `range()` é usada para gerar sequências numéricas, muito comum em repetições com `for`. Ela cria uma progressão aritmética com `início`, `fim` e `passo`.

Formas de uso:

- `range(n)`
Gera números de 0 até $n - 1$, com passo 1.
Ex: `range(5)`: [0, 1, 2, 3, 4]
- `range(i, n)`
Gera números de i até $n - 1$, com passo 1.
Ex: `range(8, 13)`: [8, 9, 10, 11, 12]
- `range(i, f, p)`
Gera números de i até antes do n , com o passo definido por p .
Ex: `range(1, 30, 5)`: [1, 6, 11, 16, 21, 26]
Ex: `range(5, -14, -3)`: [5, 2, -1, -4, -7, -10, -13]

Exemplo de uso com for:

```
1 for i in range(10):  
2     print(i, end = " ")           # saída: 0 1 2 3 4 5 6 7 8 9
```

Outro Exemplo do cálculo do Fatorial

```
1 num = int(input("Digite valor inteiro e positivo: "))
2 fat = 1
3 for i in range(1, num + 1):
4     fat = fat * i
5 print("O fatorial de", num, "=", fat)
```

Obs: O `range(1, num + 1)` percorre os valores de 1 até num, inclusive.

Variável Acumuladora

Vamos ver alguns exemplos de problemas que são resolvidos utilizando **laços**.

Há alguns padrões de solução que são bem conhecidos, e são úteis em diversas situações.

O primeiro padrão deles é o uso de uma “**variável acumuladora**”.

Problema: Ler um inteiro positivo n , depois ler n números e exibir a soma.

Por que usar uma **variável acumuladora**?

- Não sabemos quantos números o usuário vai digitar.
- Não é prático criar uma variável para cada número.
- A solução é usar uma variável que acumule a soma a cada repetição.

Padrão:

```
1 acumuladora = 0 # inicialmente nada foi somado
2 Repita n vezes:
3     Leia um número aux
4     acumuladora = acumuladora + aux
```

Exemplo: Soma de n números

Solução utilizando o laço for:

```
1 n = int(input("Digite o valor de n: "))
2 acu = 0
3 for i in range(n):
4     aux = int(input())
5     acu = acu + aux
6 print("A soma é: " + str(acu))
```

Observações:

- A variável acu começa com 0.
- A cada repetição, somamos um novo número à variável acumuladora.
- No final, imprimimos o valor total acumulado.

Exemplo: Soma com while

A mesma ideia com laço while:

```
1 n = int(input("Digite o valor de n: "))
2 acu = 0
3 i = 0
4 while i < n:
5     aux = int(input())
6     acu = acu + aux
7     i = i + 1
8 print("A soma é: " + str(acu))
```

Observações:

- Inicializamos o contador $i = 0$.
- O laço se repete até $i < n$.
- A lógica da variável acumuladora continua igual.

Calculando Potências de 2

Problema: Ler um número inteiro positivo n e imprimir as potências:
 $2^0, 2^1, 2^2, \dots, 2^n$

Estratégia:

- Usamos uma variável acumuladora que começa valendo $2^0 = 1$
- A cada iteração, imprimimos o valor atual e multiplicamos por 2

Pseudocódigo:

```
1 acumuladora = 1                # corresponde 2^0
2 Para i = 0 até n faça:
3     imprima acumuladora
4     acumuladora = acumuladora * 2
```

Potências de 2 com for

```
1 n = int(input("Digite o valor de n: "))
2 acu = 1 # corresponde a 2^0
3
4 for i in range(n + 1):
5     print("2^" + str(i) + " = " + str(acu))
6     acu = acu * 2
```

Aqui o controle do índice é automático com range(n+1).

Saída para entrada: 4

```
1 Digite o valor de n: 4
2 2^0 = 1
3 2^1 = 2
4 2^2 = 4
5 2^3 = 8
6 2^4 = 16
```

Potências de 2 com while

```
1 n = int(input("Digite o valor de n: "))
2 i = 0
3 acu = 1                                # corresponde a 2^0
4
5 while i <= n:
6     print("2^" + str(i) + " = " + str(acu))
7     i = i + 1
8     acu = acu * 2
```

Saída esperada para $n = 4$:

```
1 2^0 = 1
2 2^1 = 2
3 2^2 = 4
4 2^3 = 8
5 2^4 = 16
```

Quando usar while ou for?

while

- Usado quando a repetição depende de uma **condição lógica**.
- A quantidade de repetições pode ser **indefinida**.
- Requer controle manual da variável de repetição.

for

- Ideal quando sabemos **quantas vezes** o laço deve se repetir.
- A contagem é automática (com **range** ou **listas**).
- Mais compacto e direto para sequências numéricas.

Resumo: Use **while** quando o fim da repetição depende de uma condição variável e **for** para laços com contagem definida.

Laços e o comando break

O comando **break** encerra a execução de um laço, mesmo que a condição ainda seja verdadeira.

Exemplo com for:

```
1 for i in range(1, 11):  
2     if i >= 5:  
3         break  
4     print(i)  
5 print("Terminou o laço")
```

Saída esperada:

```
1 1  
2 2  
3 3  
4 4  
5 Terminou o laço
```

Laços e o comando break

Assim como as condições em laços, o break também define **quando o laço deve parar**.

Exemplo com while True:

```
1 i = 1
2 while True:
3     if i > 10:
4         break
5     print(i)
6     i = i + 1
```

É equivalente a:

```
1 i = 1
2 while i <= 10:
3     print(i)
4     i = i + 1
```

Laços e o comando continue

O comando `continue` pula o restante do bloco e volta para o início do laço.

Exemplo com while:

```
1 i = 1
2 while i <= 10:
3     if i == 5:
4         i = i + 1
5         continue
6     print(i)
7     i = i + 1
8 print("Terminou o laço")
```

Saída esperada: números de 1 a 10, exceto o 5.

Laços e o comando continue

Usamos continue quando só queremos executar comandos sob certas condições.

Exemplo: imprimir área de círculos com raio par entre 1 e 10

```
1 for r in range(1, 11):  
2     if r % 2 != 0:                # pula ímpares  
3         continue  
4     area = 3.1415 * r ** 2        # ou area = 3.1415 * r * r  
5     print("%.2f" % area)
```

Forma mais direta:

```
1 for r in range(2, 11, 2):  
2     area = 3.1415 * r ** 2  
3     print("%.2f" % area)
```

Menu de escolhas

Em programas de computador, é comum a apresentação de um **menu de opções** para o usuário.

Vamos fazer um menu com **algumas opções**, incluindo uma última para encerrar o programa.

Exercício: Menu com opção de saída

Faça um programa que imprima um menu com 4 pratos e uma 5ª opção para sair (fechar o programa). O programa deve exibir o prato escolhido e só termina quando a opção 5 for selecionada. Os pratos são enumerados de 1 a 4.

Solução

```
1 while True:
2     print("\nMenu:")
3     print("1 - Lasanha")
4     print("2 - Estrogonofe")
5     print("3 - Feijoada")
6     print("4 - Salada")
7     print("5 - Sair")
8
9     opcao = int(input("Escolha uma opção: "))
10
11     if opcao == 1:
12         print("Você escolheu Lasanha.")
13     elif opcao == 2:
14         print("Você escolheu Estrogonofe.")
15     elif opcao == 3:
16         print("Você escolheu Feijoada.")
17     elif opcao == 4:
18         print("Você escolheu Salada.")
19     elif opcao == 5:
20         print("Encerrando o programa...")
21         break
22     else:
23         print("Opção inválida.")
```

Variável Indicadora

Para que serve? É usada em laços para verificar se um objeto (ou conjunto de objetos) **satisfaz** uma determinada propriedade.

Padrão de uso:

- Inicializamos a variável indicadora com **True**, assumindo que a propriedade é satisfeita.
- Durante o laço, verificamos cada item.
- Se algum item violar a propriedade, alteramos a variável para **False**.

Exemplo típico: Verificar se todos os números de uma lista são positivos.

Estrutura com Variável Indicadora

Padrão de implementação:

```
1  indicadora = True # assume que a propriedade é satisfeita
2
3  for item in estrutura:
4      if not verifica_propriedade(item):
5          indicadora = False
6          break
```

Exemplo:

- Se indicadora == True, todos os itens satisfazem a condição.
- Se indicadora == False, ao menos um item violou a regra.

Exemplo com Variável Indicadora

Problema: Verificar se todos os números de uma lista são pares.

```
1 lista = [2, 4, 6, 8, 10]
2 todos_pares = True
3
4 for num in lista:
5     if num % 2 != 0:
6         todos_pares = False
7         break
8
9 if todos_pares:
10     print("Todos os números são pares.")
11 else:
12     print("Existe pelo menos um número ímpar.")
```

Obs: A variável `todos_pares` atua como uma "bandeira lógica" ao longo do laço.

Exemplo: Números Primos (lógica)

Problema: Verificar se um número n é primo.

Definição: Um número é primo se só é divisível por 1 e ele mesmo.

Ideia:

- Testar todos os divisores entre 2 e $n - 1$.
- Se algum dividir n , então não é primo.
- Usar uma variável indicadora (`eprimo = True`).

Verificação de primo com while

```
1 n = int(input("Digite um número: "))
2 div = 2
3 eprimo = True
4
5 while (div <= n - 1) and eprimo:
6     if n % div == 0:
7         eprimo = False
8         div = div + 1
9
10 if eprimo:
11     print("É primo!!")
12 else:
13     print("Não é primo!!")
```

Comando break na verificação de primos

```
1 n = int(input("Digite um número: "))
2 div = 2
3 eprimo = True
4
5 while div <= n - 1:
6     if n % div == 0:
7         eprimo = False
8         break
9     div = div + 1
10
11 if eprimo:
12     print("É primo!!")
13 else:
14     print("Não é primo!!")
```

Verificando se a sequência está ordenada

Problema: Ler n números e informar se estão em ordem crescente.

Estratégia:

- Comparar cada número com o anterior.
- Se encontrar um número menor que o anterior, a sequência não está ordenada.
- Usar uma variável indicadora ordenado = True.

Implementação em Python

```
1 n = int(input("Digite um número: "))
2 anterior = int(input())
3 i = 1
4 ordenado = True
5
6 while (i < n) and ordenado:
7     atual = int(input())
8     i = i + 1
9     if atual < anterior:
10         ordenado = False
11         anterior = atual
12
13 if ordenado:
14     print("Sequência está ordenada")
15 else:
16     print("Sequência não está ordenada")
```

Verificando primos com variável contadora

Ideia: Contar quantos números dividem n além de 1 e ele mesmo.

```
1 n = int(input("Digite um número: "))
2 div = 2
3 divisores = 0
4
5 while div <= n - 1:
6     if n % div == 0:
7         divisores += 1
8     div += 1
9
10 if divisores == 0:
11     print("É primo!!")
12 else:
13     print("Não é primo!! Há", divisores, "divisores de", n)
```

Com otimização: parar ao encontrar divisor

```
1 n = int(input("Digite um número: "))
2 div = 2
3 divisores = 0
4
5 while (div <= n - 1) and (divisores == 0):
6     if n % div == 0:
7         divisores += 1
8     div += 1
9
10 if divisores == 0:
11     print("É primo!!")
12 else:
13     print("Não é primo!!")
```

Acumuladora, Indicadora e Contadora

Padrões úteis em laços:

- **Acumuladora:** Soma ou multiplica valores. Ex: soma de notas, cálculo de fatorial.
- **Contadora:** Conta quantas vezes algo ocorre. Ex: número de divisores, ocorrências de um caractere.
- **Indicadora:** Registra se uma condição foi ou não satisfeita. Ex: todos positivos, sequência ordenada, número primo.

Esses padrões podem aparecer juntos ou separados, dependendo do problema.

Exemplo: Encontrar o maior número

Problema: Ler n números e imprimir o maior.

Ideia:

- Ler o primeiro número e armazenar em maior.
- Comparar os próximos e atualizar se encontrar um maior.

Implementação do maior número

```
1 n = int(input("Digite um número: "))
2 maior = int(input())
3 cont = 1
4
5 while cont < n:
6     aux = int(input())
7     if aux > maior:
8         maior = aux
9     cont += 1
10
11 print("O maior número é:", maior)
```

Números de Fibonacci

A série de Fibonacci é uma sequência onde cada termo é a soma dos dois anteriores:

$$F(n) = F(n-1) + F(n-2), \quad \text{com } F(1) = 1 \text{ e } F(2) = 1$$

Exemplo da sequência:

1, 1, 2, 3, 5, 8, 13, 21, ...

Problema: Fazer um programa que imprime os primeiros n números da série de Fibonacci.

Lógica:

- Começamos com $f_{\text{ant}} = 0$ e $f_{\text{atual}} = 1$.
- A cada passo, imprimimos f_{atual} , atualizamos os valores e repetimos até imprimir n termos.

Fibonacci com while

```
1 n = int(input("Digite um número: "))
2 cont = 0
3 f_ant = 0
4 f_atual = 1
5
6 while cont < n:
7     print(f_atual, ", ", ", end="") # imprime o número
8     cont = cont + 1                # conta mais um
9     f_aux = f_atual                # guarda o valor atual
10    f_atual = f_atual + f_ant       # atualiza f_atual
11    f_ant = f_aux                  # atualiza f_ant
```

Saída esperada para $n = 6$:

```
1 1, 1, 2, 3, 5, 8,
```

Laços Encaixados

- Um laço (for ou while) pode conter outro laço em seu bloco.
- Essa estrutura é útil para trabalhar com dados organizados em pares, matrizes, tabelas, etc.
- Cada vez que o laço externo dá um passo, o laço interno executa completamente.

Exemplo:

```
1 for i in range(1, 11):  
2     for j in range(1, 6):  
3         print(i, j)
```

Saída (parcial):

```
1 1 1  
2 1 2  
3 1 3  
4 1 4  
5 1 5  
6 2 1  
7 ...  
8 10 5
```

Entendendo os Laços Encaixados

- O laço externo (i) controla as repetições maiores.
- O laço interno (j) reinicia a cada iteração do externo.
- No exemplo: para cada i de 1 a 10, j vai de 1 a 5.

Total de linhas impressas: $10 \times 5 = 50$

Esse padrão é comum em algoritmos que processam linhas e colunas, ou combinações de pares de valores.

Exemplo: Tabuada com Laços Encaixados

```
1 # Tabuada de 1 a 5
2 for i in range(1, 6):
3     print(f"Tabuada do {i}:")
4     for j in range(1, 11):
5         print(f"{i} x {j} = {i*j}")
6     print() # linha em branco entre tabuadas
```

Saída (parcial):

```
1 Tabuada do 1:
2 1 x 1 = 1
3 1 x 2 = 2
4 ...
5 1 x 10 = 10
6
7 Tabuada do 2:
8 2 x 1 = 2
9 ...
10 2 x 2 = 4
11 .
12 .
13 .
```

Laços Encaixados: Todos os Resultados com 4 Dados

- Problema: imprimir todas as combinações possíveis ao jogar 4 dados de 6 faces.
- Solução: usar **4 laços for aninhados**, um para cada dado.
- Cada laço representa um dado e vai de 1 a 6.

Código em Python:

```
1 print("D1 D2 D3 D4")
2 for d1 in range(1, 7):
3     for d2 in range(1, 7):
4         for d3 in range(1, 7):
5             for d4 in range(1, 7):
6                 print(d1, d2, d3, d4)
```

Total de combinações: $6^4 = 1296$